PROJETO DE BANCO DE DADOS

Sistema Musical: Catálogo de uma Gravadora

Trabalho – Banco de Dados II (versão atualizada)

Alunos

Gabriel Ribeiro de Oliveira
Ithalo Cavalcante Gomes
Wilker Oliveira dos Santos

1. Introdução

Este trabalho apresenta o banco de dados e o sistema de um catálogo musical de gravadora, que organiza cantores, perfis, álbuns, gêneros e músicas. O banco foi implementado em SQLite e o sistema em Python, com orientação a objetos e interface web em Flask.

Em relação à proposta inicial, a implementação concentrou-se no catálogo musical e incluiu o controle de acesso de usuários. As diferenças entre a proposta e a versão atual estão resumidas na seção 3.1.

2. Objetivos

O objetivo geral é criar uma estrutura que armazene e relacione os dados do catálogo de uma gravadora, e um sistema que consulte e mantenha esses dados. De forma específica, o projeto deve:

- cadastrar cantores, álbuns e gêneros musicais, com um perfil complementar opcional para cada cantor;
- organizar as músicas por álbum e por gênero e registrar quais cantores participam de cada uma;
- garantir a consistência dos dados por meio de chaves primárias, chaves estrangeiras e restrições;
- implementar em classes Python as operações de seleção, inserção, atualização e remoção;
- oferecer consultas que juntam várias tabelas e aplicam filtros de condições;
- separar quem apenas consulta o catálogo de quem pode alterá-lo.

3. Estrutura do banco

O modelo é formado por sete tabelas: cinco entidades do domínio, uma tabela de junção e uma tabela de usuários.

Tabela	Função	Principais campos
cantor	Artistas.	nome, data de nascimento, nacionalidade
perfil_cantor	Dados complementares do cantor.	biografia, instagram, foto
album	Lançamentos do catálogo.	nome, ano de lançamento, gravadora

Tabela	Função	Principais campos
genero	Classificação musical das faixas.	nome, descrição
musica	Faixas dos álbuns.	título, duração, data de lançamento, álbum, gênero
cantor_musica	Participação dos cantores nas músicas (tabela de junção).	cantor, música
usuario	Contas de acesso ao sistema.	nome, e-mail, senha (hash), papel

3.1 Mudanças em relação à proposta

A proposta previa oito tabelas em MySQL. A versão implementada simplificou o modelo para o catálogo e acrescentou o controle de acesso:

Proposta	Versão atual
Banco em MySQL	Banco em SQLite, em um único arquivo (musical.db)
Tabelas Gravadora e Contrato	Removidas; a gravadora passou a ser um campo de texto do álbum
Álbum ligado a um cantor (Album.id_cantor)	Removido; o cantor se liga ao álbum pelas músicas, via cantor_musica
Campos nome_artistico, tipo, numero_faixa e participacao	Não fazem parte da versão atual
Perfil_Cantor com id_cantor como chave primária	perfil_cantor com chave própria (idPerfil) e idCantor único
Tipos VARCHAR, DATE, TIME e DECIMAL	Tipos TEXT e INTEGER do SQLite; datas e duração guardadas como texto
Sem controle de acesso	Tabela usuario, com senha em hash e papéis admin e usuario

4. Modelo de dados e relacionamentos

As tabelas se conectam da seguinte forma:

Relacionamento	Cardinalidade	Como é implementado
cantor – perfil_cantor	1:1	perfil_cantor.idCantor é chave estrangeira com UNIQUE
album – musica	1:N	musica.idAlbum (obrigatório)
genero – musica	1:N	musica.idGenero (obrigatório)
cantor – musica	N:N	tabela cantor_musica, com chave primária composta (idCantor, idMusica)

A tabela usuario não se relaciona com as demais: ela controla quem acessa o sistema e não faz parte do catálogo.

4.1 Perfil do cantor

perfil_cantor guarda os dados que nem todo cantor possui: biografia, Instagram e foto. A tabela tem chave própria (idPerfil), e o idCantor é chave estrangeira com UNIQUE, o que impede dois perfis para o mesmo cantor e garante o 1:1. Instagram e foto são opcionais, e a foto é armazenada como caminho ou URL do arquivo.

4.2 Álbuns, gêneros e músicas

Cada música pertence a exatamente um álbum e a um gênero, por isso idAlbum e idGenero são obrigatórios em musica. A gravadora é registrada como texto no próprio álbum e é opcional.

A duração é guardada no formato HH:MM:SS e as datas no formato AAAA-MM-DD, como texto. O SQLite não tem um tipo próprio para datas, e suas funções de data, como strftime, trabalham sobre texto nesse formato.

4.3 Participação dos cantores

cantor_musica resolve o N:N entre cantores e músicas: uma música pode ter vários cantores, e um cantor pode ter várias músicas. A chave primária é composta pelas duas chaves estrangeiras, o que impede vincular o mesmo cantor duas vezes à mesma música. Por exemplo, “Águas de Março” está ligada a Elis Regina e a Tom Jobim.

4.4 Usuários

usuario guarda as contas de acesso: o e-mail não se repete, a senha é armazenada apenas como hash e o papel é admin ou usuario. O cadastro feito pelo site sempre cria o papel usuario, que só consulta. O administrador é criado apenas pelo script local configurar_admin.py.

5. Regras de integridade

Além das chaves primárias e estrangeiras, o banco aplica as seguintes restrições:

Regra	Onde é aplicada
As chaves estrangeiras são verificadas em toda conexão.	PRAGMA foreign_keys = ON, em banco.py
Um cantor tem no máximo um perfil.	UNIQUE em perfil_cantor.idCantor
Toda música tem álbum e gênero.	NOT NULL em musica.idAlbum e musica.idGenero
O mesmo cantor não é vinculado duas vezes à mesma música.	Chave primária composta em cantor_musica
Álbum ou gênero com músicas não pode ser excluído.	ON DELETE RESTRICT em musica.idAlbum e musica.idGenero
Excluir um cantor remove seu perfil e seus vínculos com músicas.	ON DELETE CASCADE em perfil_cantor.idCantor e cantor_musica.idCantor

Regra	Onde é aplicada
Excluir uma música remove seus vínculos com cantores.	ON DELETE CASCADE em cantor_musica.idMusica
O e-mail não se repete, sem diferenciar maiúsculas.	UNIQUE e COLLATE NOCASE em usuario.email
Papel limitado a admin ou usuario.	CHECK em usuario.papel (padrão: usuario)
Nomes, datas e chaves são obrigatórios.	NOT NULL; só instagram, foto, gravadora e descricao aceitam NULL

6. Diagramas

O banco está representado em dois níveis, e o sistema em um terceiro diagrama: o modelo relacional mostra as entidades, os atributos e os relacionamentos; o modelo lógico mostra as tabelas como foram criadas, com tipos e chaves; e o diagrama de classes mostra como as tabelas viraram classes Python.

6.1 Modelo relacional (DER)

O diagrama reúne as seis entidades — Cantor, Perfil_Cantor, Album, Genero, Musica e Usuario — na notação de Chen: entidades em retângulos, atributos em elipses, com a chave primária sublinhada, e relacionamentos em losangos, com a cardinalidade em cada ligação. Os relacionamentos são POSSUI (1:1), CANTA (N:N), CONTÉM (1:N) e CLASSIFICA (1:N). Usuario aparece sem relacionamentos, porque não faz parte do catálogo.

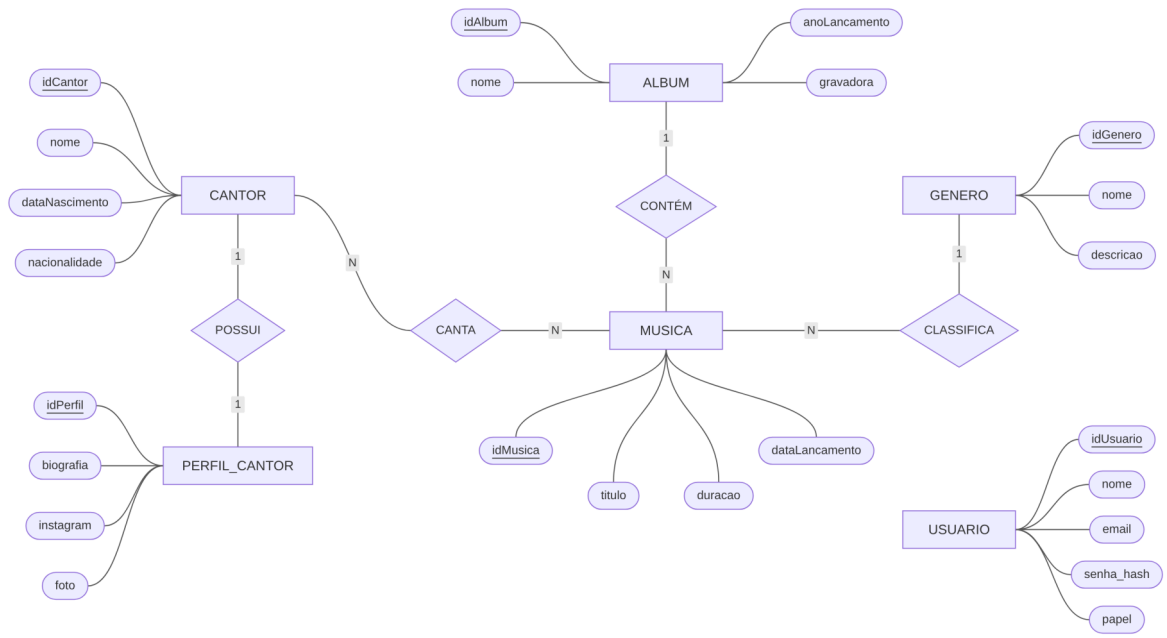


Figura 1 – Modelo relacional: entidades, atributos e relacionamentos do catálogo.

6.2 Modelo lógico

Aplicadas as regras de conversão, o diagrama acima resulta nas sete tabelas abaixo, com seus campos, tipos, chaves e regras de exclusão. O relacionamento N:N CANTA virou a tabela cantor_musica; os relacionamentos 1:N viraram chaves estrangeiras no lado N (musica); e o 1:1 virou uma chave estrangeira única em perfil_cantor.

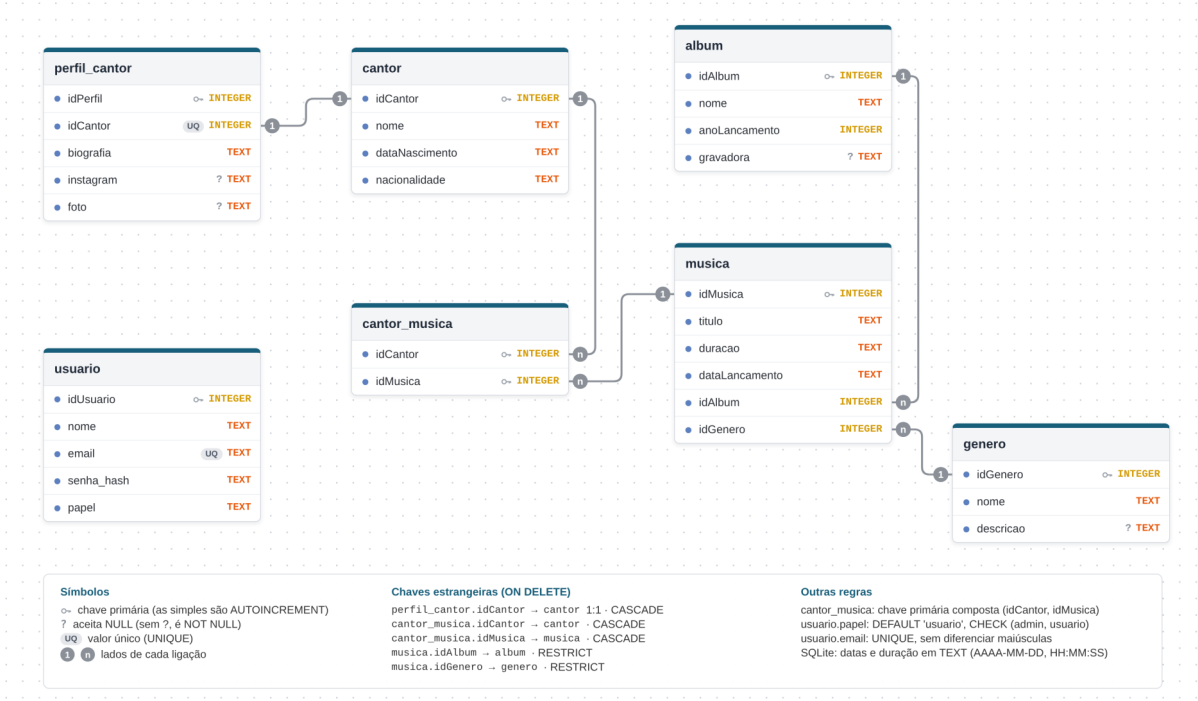


Figura 2 – Modelo lógico: tabelas, tipos, chaves e regras de exclusão.

6.3 Diagrama de classes

Cada tabela de entidade virou uma classe Python. As cinco classes do catálogo herdam de ModeloBase, que concentra o CRUD. Usuario fica separada, com métodos próprios, para não expor o hash das senhas nem aceitar o papel vindo do formulário. Na notação, os métodos sublinhados são

de classe (estáticos), os atributos com “/” são derivados, calculados nas consultas, e o sinal “-” indica atributo privado.

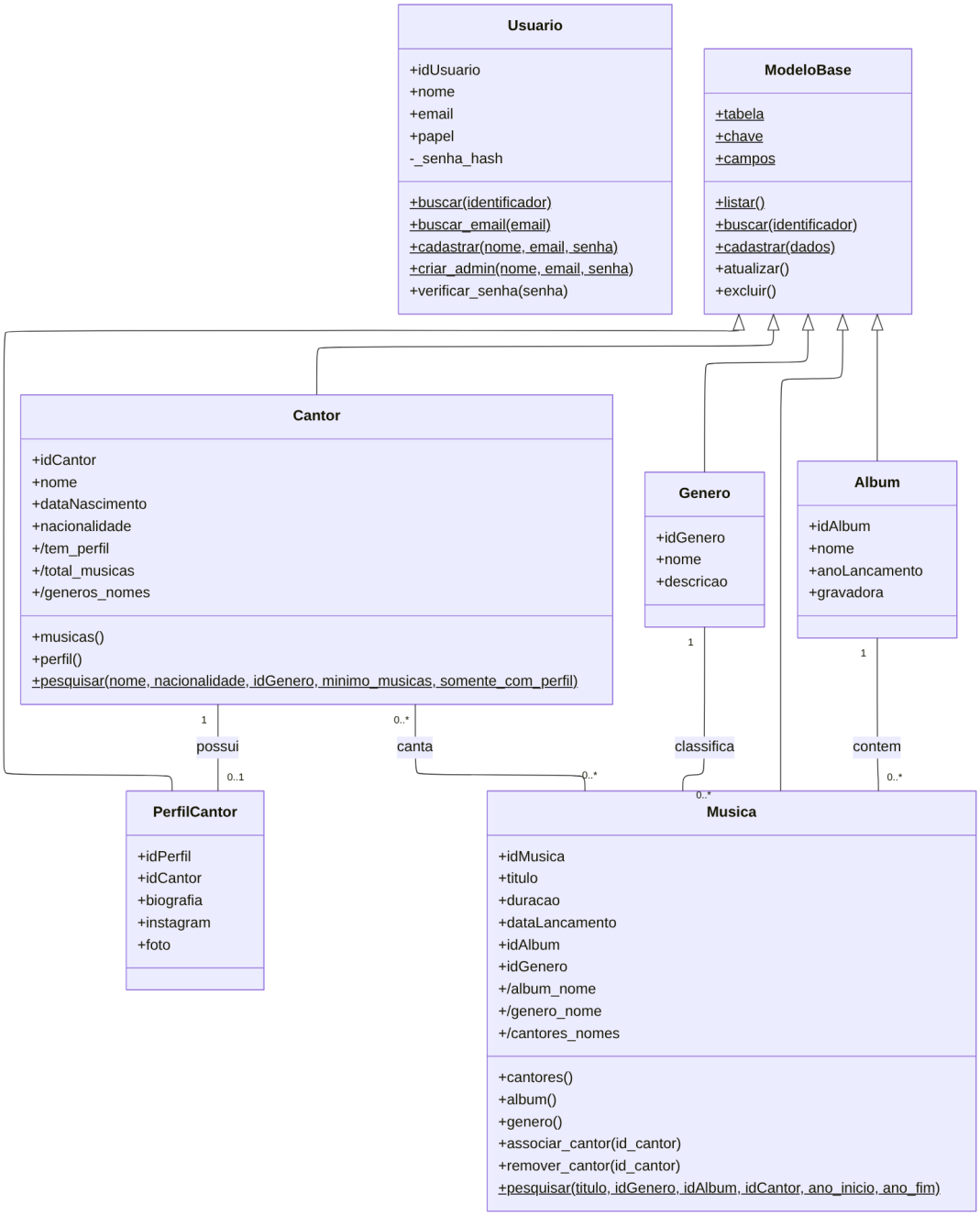


Figura 3 – Diagrama de classes do sistema.

7. Implementação

7.1 Organização do código

Arquivo ou pasta	Função
banco.sql e banco.py	Criação das tabelas e conexão com o SQLite
modelos/	Classes do sistema, com as regras e o SQL

Arquivo ou pasta	Função
app.py	Rotas do Flask: recebe as requisições e chama as classes
templates/ e static/	Telas em HTML e estilo
configurar_admin.py	Criação local do administrador
popular_banco.py	Cadastro de dados de exemplo para demonstração

7.2 Operações de CRUD

Cada classe do catálogo informa apenas o nome da tabela, a chave e a lista de campos. A partir disso, a ModeloBase monta o SQL de cada operação, e cada linha devolvida pelo banco vira um objeto da classe correspondente:

Operação	Método	Comando SQL
Inserção	`cadastrar()`	INSERT
Seleção	`listar()` e `buscar()`	SELECT
Atualização	`atualizar()`	UPDATE
Remoção	`excluir()`	DELETE

Os valores sempre entram como parâmetros (?), separados do comando SQL, o que evita injeção de SQL. Os relacionamentos também viraram métodos: `cantor.perfil()`, `cantor.musicas()`, `musica.album()`, `musica.genero()` e `musica.cantores()`, além de `associar_cantor()` e `remover_cantor()`, que gravam e apagam vínculos em cantor_musica.

7.3 Controle de acesso

O visitante só cria conta e faz login. O usuário comum consulta e filtra o catálogo, e o administrador cadastra, edita, exclui e vincula cantores às músicas. A verificação é feita nas próprias rotas, pelos decorators login_obrigatorio e admin_obrigatorio, e não apenas escondendo botões. Os formulários que alteram dados usam token CSRF.

8. Consultas com junção e filtros

Além das seleções simples do CRUD, o sistema tem duas seleções complexas. Elas juntam várias tabelas e aplicam apenas os filtros que o usuário preencheu: cada filtro preenchido acrescenta uma condição ao WHERE.

8.1 Busca de músicas

`Musica.pesquisar()` junta musica, album, genero, cantor_musica e cantor e devolve uma linha por música, com os nomes do álbum, do gênero e dos cantores no lugar dos ids. Álbum e gênero usam JOIN, porque toda música tem os dois; os cantores usam LEFT JOIN, para que uma música ainda sem cantor não desapareça. O GROUP BY e o GROUP_CONCAT juntam numa única linha uma música com vários cantores.

Os filtros são título (LIKE), gênero, álbum, cantor e intervalo de anos de lançamento. O filtro de cantor usa uma subconsulta, para que a música continue mostrando todos os seus cantores. Com os filtros de gênero e ano preenchidos, a consulta fica assim:

```
SELECT m.*, a.nome AS album_nome, g.nome AS genero_nome,
       GROUP_CONCAT(c.nome, ', ') AS cantores_nomes
FROM musica m
JOIN album a  ON a.idAlbum  = m.idAlbum
JOIN genero g ON g.idGenero = m.idGenero
LEFT JOIN cantor_musica cm ON cm.idMusica = m.idMusica
LEFT JOIN cantor c ON c.idCantor = cm.idCantor
WHERE m.idGenero = ?
  AND CAST(strftime('%Y', m.dataLancamento) AS INTEGER) >= ?
GROUP BY m.idMusica
ORDER BY m.titulo
```

Com os dados de exemplo, o gênero Rock a partir de 1975 retorna '39, Bohemian Rhapsody e Love of My Life, todas do álbum A Night at the Opera.

8.2 Busca de cantores

`Cantor.pesquisar()` usa os três tipos de relacionamento na mesma consulta: o perfil (1:1), as músicas (N:N) e os gêneros das músicas (1:N). Todas as junções são LEFT JOIN, para manter os cantores sem perfil ou sem músicas, e o COUNT conta as músicas de cada cantor.

Nome, nacionalidade, gênero e “somente com perfil” filtram linhas e ficam no WHERE. O mínimo de músicas depende da contagem, que só existe depois do GROUP BY, e por isso fica no HAVING:

```
SELECT c.*,
       CASE WHEN p.idPerfil IS NOT NULL THEN 1 ELSE 0 END AS tem_perfil,
       COUNT(DISTINCT m.idMusica) AS total_musicas,
       GROUP_CONCAT(DISTINCT g.nome) AS generos_nomes
FROM cantor c
LEFT JOIN perfil_cantor p  ON p.idCantor = c.idCantor
LEFT JOIN cantor_musica cm ON cm.idCantor = c.idCantor
LEFT JOIN musica m         ON m.idMusica = cm.idMusica
LEFT JOIN genero g         ON g.idGenero = m.idGenero
WHERE c.nacionalidade LIKE ?
  AND p.idPerfil IS NOT NULL
GROUP BY c.idCantor
HAVING COUNT(DISTINCT m.idMusica) >= ?
ORDER BY c.nome
```

Com os dados de exemplo, a nacionalidade “brasileira”, somente com perfil e com no mínimo duas músicas, retorna Elis Regina e Marisa Monte.

9. Conclusão

A versão atual organiza em sete tabelas relacionadas o catálogo de uma gravadora, do cadastro de cantores até a composição de cada álbum, e o disponibiliza em um sistema web com controle de acesso.

Com isso, o projeto aplica na prática os conceitos de entidades, relacionamentos 1:1, 1:N e N:N, chaves, integridade referencial, orientação a objetos e consultas com junção e filtros estudados na disciplina.

Link do repositório do git: https://github.com/gabriel-gbl/projeto-bdd-2